HossainMohammad

Shahjalal University of Science and Technology

1 Data Structure **1**
- 1.1 Fenwick Tree . . . 1
- 1.2 Fenwick Tree 2D Index Compression . . . 1
- 1.3 Fenwick 2D . . . 1
- 1.4 Lazy Propagation . . . 1
- 1.5 Persistent Segment Tree . . . 1
- 1.6 Persistent Lazy Propagation 2
- 1.7 Segment Tree 2D . . . 2
- 1.8 Segment Tree 2D (Index compression) . . . 2
- 1.9 Merse Sort Tree with Update 2
- 1.10 Treap . . . 3
- 1.11 Persistent DSU . . . 3
- 1.12 MOs Algorithm . . . 3
- 1.13 MOs with updates . . . 3
- 1.14 MOs on Trees . . . 3
- 1.15 Dynamic Connectivity . . . 3
- 1.16 Xor Basis . . . 4
- 1.17 Link Cut Tree . . . 4
- 1.18 Wavelet Tree . . . 4
- 1.19 Sparse Table (RMQ) . . . 4

2 Geometry **4**
- 2.1 Geometry Primitives . . . 4
- 2.2 Point 2D . . . 5
- 2.3 Polar Sort . . . 5
- 2.4 Line Intersection . . . 5
- 2.5 Line Equation . . . 5
- 2.6 Segment Distance . . . 5
- 2.7 Segment Intersection . . . 5
- 2.8 Side Of . . . 5
- 2.9 On Segment . . . 5
- 2.10 Linear Transformation . . . 5
- 2.11 Circle Intersection . . . 5
- 2.12 Circle Polygon Intersection . 5
- 2.13 Circle Line Intersection . . . 5
- 2.14 Circle Segment Intersection . 5
- 2.15 Circle Tangents . . . 6
- 2.16 Minimum Enclosing Circle . 6
- 2.17 Inside Polygon . . . 6
- 2.18 Polygon Area . . . 6
- 2.19 Polygon Center . . . 6
- 2.20 Polygon Cut . . . 6
- 2.21 Polygon Union . . . 6
- 2.22 Extreme Vertex . . . 6
- 2.23 Manhattan MST . . . 6
- 2.24 Convex Hull . . . 6
- 2.25 Hull Diameter . . . 7
- 2.26 Point Inside Hull . . . 7
- 2.27 Line Hull Intersection . . . 7
- 2.28 Half Plane Intersection . . . 7
- 2.29 Minkowski Sum Dif . . . 7
- 2.30 Maximum Inscribed Circle . 7
- 2.31 Rotating Calipers . . . 7
- 2.32 Closest Pair . . . 7

3 Geometry 3D **7**
- 3.1 Point 3D . . . 7
- 3.2 Plane . . . 8
- 3.3 Line 3D . . . 8
- 3.4 Plane Line . . . 8
- 3.5 Convex Hull 3d . . . 8
- 3.6 Sphere . . . 9
- 3.7 TetraHedron Volume . . . 9

1 Data Structure

1.1 Fenwick Tree

Description: Use for point update + range query or range update + point query. **1-indexed**.
Time: $O(\log N)$

```
struct FT {
  int n; vector<ll> s;
  FT(int n) : n(n + 1), s(n + 1) {}
  void update(int pos, ll dif) {
    for (; pos < n; pos += pos & -pos) s
     [pos] += dif;
  }
  ll query(int pos) {
    ll res = 0;
    for (; pos > 0; pos -= pos & -pos)
     res += s[pos];
    return res;
  }
  ll query(int l, int r) {
    return query(r) - query(l - 1);
  }
  void update(int l, int r, ll dif) {
    update(l, dif);
    if (r + 1 < n) update(r + 1, -dif);
  }
  int lower_bound(ll sum) {
    if (sum <= 0) return 0;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1)
     {
      if (pos + pw < n && s[pos+pw] <
     sum) {
        pos |= pw, sum -= s[pos];
      }
    }
    return pos + 1;
  }
};
```

1.2 Fenwick Tree 2D Index Compression

Description: Computes sums $a[i,j]$ for all $i \le I, j \le J$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call `fakeUpdate()` before `init()`).
Time: $O(\log^2 N)$. (Use persistent segment trees for $O(\log N)$)

```
struct FT2 {
  int n; vector<FT> ft;
  vector<vector<int>> ys;
  FT2(int limx) : n(limx + 1), ys(limx +
      1) {}
  void fakeUpdate(int x, int y) {
    for (; x < n; x += x & -x) {
      ys[x].push_back(y);
    }
  }
  void init() {
    ft.assign(ys.size(), FT(0));
    for (int i = 0; i < n; ++i) {
      sort(ys[i].begin(), ys[i].end());
      ys[i].erase(unique(ys[i].begin(),
     ys[i].end()), ys[i].end());
      ft[i] = FT(ys[i].size());
    }
  }
  int ind(int x, int y) {
    return upper_bound(ys[x].begin(), ys
     [x].end(), y) - ys[x].begin();
  }
  void update(int x, int y, ll dif) {
    for (; x < n; x += x & -x)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x > 0; x -= x & -x) {
      sum += ft[x].query(ind(x, y));
    }
    return sum;
  }
  ll query(int x1, int y1, int x2, int
     y2) {
    return query(x2,y2) - query(x1-1, y2
     ) -
        query(x2, y1-1) + query(x1-1, y1
     -1);
  }
};
```

1.3 Fenwick 2D

```
struct FT2D {
  int n, m; vector<vector<ll>> s;
  FT2D(int n, int m) : n(n + 1), m(m +
     1), s(n + 1, vector<ll>(m + 1)) {}
  FT2D(int n) : FT2D(n, n) {}
  void update(int x, int y, ll dif) {
    for (int i = x; i < n; i += i & -i)
      for (int j = y; j < m; j += j & -j
     ) s[i][j] += dif;
  }
  ll query(int x, int y) {
    ll res = 0;
    for (int i = x; i > 0; i -= i & -i)
      for (int j = y; j > 0; j -= j & -j
     ) res += s[i][j];
    return res;
  }
  ll query(int x1, int y1, int x2, int
     y2) {
    return query(x2, y2) - query(x1 - 1,
      y2) - query(x2, y1 - 1) + query(x1
      - 1, y1 - 1);
  }
};
```

1.4 Lazy Propagation

```
struct ST {
  #define lc (n << 1)
  #define rc ((n << 1) | 1)
  vector<ll> a, t, lazy;
  ST(vector<ll>& a) : a(a) {
    lazy.resize(4 * a.size() + 1, 0);
    t.resize(4 * a.size() + 1, 0);
  }
  ST(int n) {
    a.resize(n);
    lazy.resize(4 * n, 0);
    t.resize(4 * n, 0);
  }
  inline long long combine(long long a,
     long long b) {
    return (a + b) % mod;
  }
  inline void pull(int n) {
    t[n] = (t[lc] + t[rc]) % mod;
  }
  // Updates n and lazy directly
  inline void apply(int n, int b, int e,
      long long v) {
    v %= mod;
    lazy[n] = (lazy[n] + v) % mod;
    t[n] = (t[n] + (ll)(e - b + 1) * v)
     % mod;
  }
  inline void push(int n, int b, int e)
     {
    if (lazy[n] == 0) return;
    if (b != e) {
      int mid = (b + e) >> 1;
      apply(lc, b, mid, lazy[n]);
      apply(rc, mid + 1, e, lazy[n]);
    }
    lazy[n] = 0;
  }
  void build(int n, int b, int e) {
    lazy[n] = 0;
    if (b == e) {
      t[n] = a[b] % mod;
      return;
    }
```

```
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
    pull(n);
  }
  void upd(int n, int b, int e, int i,
     int j, long long v) {
    if (j < b || e < i) return;
    if (i <= b && e <= j) {
      apply(n, b, e, v);
      return;
    }
    push(n, b, e);
    int mid = (b + e) >> 1;
    upd(lc, b, mid, i, j, v);
    upd(rc, mid + 1, e, i, j, v);
    pull(n);
  }
  long long query(int n, int b, int e,
     int i, int j) {
    if (i > e || b > j) return 0;
    if (i <= b && e <= j) return t[n];
    push(n, b, e);
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, i,
     j), query(rc, mid + 1, e, i, j));
  }
};
```

1.5 Persistent Segment Tree

```
struct PST {
#define lc t[cur].l
#define rc t[cur].r
  struct node {
    int l = 0, r = 0, val = 0;
  } t[40 * N];
  int T = 0;
  int combine(int a, int b) { return a +
      b; } // change
  void pull(int cur) { t[cur].val =
     combine(t[lc].val, t[rc].val); }
  int build(int b, int e) {
    int cur = ++T;
    if (b == e) return cur;
    int mid = b + e >> 1;
    lc = build(b, mid);
    rc = build(mid + 1, e);
    pull(cur);
    return cur;
  }
  // Build tree from initial vector
  int build(int b, int e, const vector<
     int>& a) {
    int cur = ++T;
    if (b == e) {
      t[cur].val = a[b];
      return cur;
    }
    int mid = b + e >> 1;
    lc = build(b, mid, a);
    rc = build(mid + 1, e, a);
    pull(cur);
    return cur;
  }
  int upd(int pre, int b, int e, int i,
     int v) {
    int cur = ++T;
    t[cur] = t[pre];
    if (b == e) {
      t[cur].val = v; // += for adding
      return cur;
    }
    int mid = b + e >> 1;
    if (i <= mid) lc = upd(t[pre].l, b,
     mid, i, v);
    else rc = upd(t[pre].r, mid + 1, e,
     i, v);
    pull(cur);
    return cur;
  }
  // Batch update: applies a sorted
     vector of updates {index, delta} in
     range [ql, qr]
  int upd_batch(int pre, int b, int e,
     const vector<pair<int, int>>& up,
     int ql, int qr) {
    if (ql > qr) return pre;
    int cur = ++T;
    t[cur] = t[pre];
    if (b == e) {
      for (int i = ql; i <= qr; ++i) t[
     cur].val += up[i].second;
      return cur;
    }
    int mid = b + e >> 1;
    int sp = upper_bound(up.begin() + ql
     , up.begin() + qr + 1, mid,
      [](int m, const pair<int, int>& p)
      { return m < p.first; }) - up.
     begin();
    lc = upd_batch(t[pre].l, b, mid, up,
      ql, sp - 1);
    rc = upd_batch(t[pre].r, mid + 1, e,
      up, sp, qr);
    pull(cur);
    return cur;
  }
  // Range query on a single version
  int query(int cur, int b, int e, int l
     , int r) {
    if (!cur || l > e || r < b) return
     0; // Identity value (0 for sum, INF
      for min)
    if (l <= b && e <= r) return t[cur].
     val;
    int mid = b + e >> 1;
    return combine(query(lc, b, mid, l,
     r), query(rc, mid + 1, e, l, r));
  }
  // Query k-th smallest element between
      versions pre and cur
  int query_kth(int pre, int cur, int b,
     int e, int k) {
    if (b == e) return b;
    int cnt = t[lc].val - t[t[pre].l].
     val;
    int mid = b + e >> 1;
    if (cnt >= k) return query_kth(t[pre
     ].l, lc, b, mid, k);
    else return query_kth(t[pre].r, rc,
     mid + 1, e, k - cnt);
  }
#undef lc
#undef rc
};
```

1.6 Persistent Lazy Propagation

```
struct PST {
#define lc t[cur].l
#define rc t[cur].r
  struct node {
    int l = 0, r = 0;
    ll val = 0, lazy = 0;
  } t[250 * N];
  int T = 0;
  inline ll combine(ll a, ll b) {
    return a + b;
  }
  inline void pull(int cur) {
    t[cur].val = combine(t[lc].val, t[rc
     ].val);
  }
  int build(int b, int e) {
    int cur = ++T;
    if(b == e) return cur;
    int mid = b + e >> 1;
    lc = build(b, mid);
    rc = build(mid + 1, e);
    pull(cur);
    return cur;
  }
  void apply(int cur, int b, int e, ll v
     ) {
    t[cur].lazy += v;
    t[cur].val += (ll)(e - b + 1) * v;
  }
  void push(int cur, int b, int e) {
    if(!t[cur].lazy) return;
    if(b != e) {
      int mid = b + e >> 1;
      int nlc = ++T; t[nlc] = t[lc];
      int nrc = ++T; t[nrc] = t[rc];
      lc = nlc; rc = nrc;
      apply(lc, b, mid, t[cur].lazy);
      apply(rc, mid + 1, e, t[cur].lazy)
      ;
    }
    t[cur].lazy = 0;
  }
  int upd(int pre, int b, int e, int i,
     int j, ll v) {
    int cur = ++T;
    t[cur] = t[pre];
    if(i <= b && e <= j) {
      apply(cur, b, e, v);
      return cur;
    }
    push(cur, b, e);
    int mid = b + e >> 1;
    if(i <= mid) lc = upd(lc, b, mid, i,
      j, v);
    if(j > mid) rc = upd(rc, mid + 1, e,
      i, j, v);
    pull(cur);
    return cur;
  }
  // Propagating query
  ll query(int cur, int b, int e, int i,
     int j) {
    if(!cur || b > j || e < i) return 0;
    if(i <= b && e <= j) return t[cur].
     val;
    push(cur, b, e);
    int mid = b + e >> 1;
    return combine(query(lc, b, mid, i,
     j), query(rc, mid + 1, e, i, j));
  }
  // Non-propagating query
  ll query(int cur, int b, int e, int i,
     int j, ll add = 0) {
    if(!cur || b > j || e < i) return 0;
    if(i <= b && e <= j) return t[cur].
     val + (ll)(e - b + 1) * add;
    int mid = (b + e) >> 1;
    add += t[cur].lazy;
    return combine(query(lc, b, mid, i,
     j, add), query(rc, mid + 1, e, i, j
     , add));
  }
#undef lc
#undef rc
};
```

1.7 Segment Tree 2D

```
// USAGE
ST2D sg(NX, NY); // dimensions 1..NX,
    1..NY
sg.upd   (1, 1, NX, x, y, v);
sg.query (1, 1, NX, x1, x2, y1, y2);
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  #define lcy (ndy << 1)
  #define rcy ((ndy << 1) | 1)
  int NX, NY;
  vector<vector<long long>> t;
  ST2D(int NX, int NY) : NX(NX), NY(NY),
      t(4 * NX, vector<long long>(4 * NY
     , 0)) {}
  inline long long combine(long long a,
     long long b) { return max(a, b); }
  inline void pull_y(int nd, int ndy) {
    t[nd][ndy] = combine(t[nd][lcy], t[
     nd][rcy]);
  }
  inline void pull_x(int nd, int ndy) {
    t[nd][ndy] = combine(t[lc][ndy], t[
     rc][ndy]);
  }
  void upd_y(int nd, int ndy, int by,
     int ey, int j, long long v, bool
     leaf_x) {
    if (by == ey) {
      if (leaf_x) t[nd][ndy] = v;
      else        t[nd][ndy] = combine(t
     [lc][ndy], t[rc][ndy]);
      return;
    }
    int mid = (by + ey) >> 1;
    if (j <= mid) upd_y(nd, lcy, by, mid
     , j, v, leaf_x);
```

```
    else          upd_y(nd, rcy, mid+1,
     ey, j, v, leaf_x);
    pull_y(nd, ndy);
  }
  long long query_y(int nd, int ndy, int
     by, int ey, int y1, int y2) {
    if (y1 > ey || by > y2) return 0;
    if (y1 <= by && ey <= y2) return t[
     nd][ndy];
    int mid = (by + ey) >> 1;
    return combine(query_y(nd, lcy, by,
     mid, y1, y2),
                   query_y(nd, rcy, mid
     +1, ey, y1, y2));
  }
  void upd(int nd, int bx, int ex, int i
     , int j, long long v) {
    if (bx == ex) {
      upd_y(nd, 1, 1, NY, j, v, true);
      return;
    }
    int mid = (bx + ex) >> 1;
    if (i <= mid) upd(lc, bx, mid, i, j,
      v);
    else          upd(rc, mid+1, ex, i,
     j, v);
    upd_y(nd, 1, 1, NY, j, v, false);
     // pull x into inner nodes
  }
  long long query(int nd, int bx, int ex
     , int x1, int x2, int y1, int y2) {
    if (x1 > ex || bx > x2) return 0;
    if (x1 <= bx && ex <= x2) return
     query_y(nd, 1, 1, NY, y1, y2);
    int mid = (bx + ex) >> 1;
    return combine(query(lc, bx, mid, x1
     , x2, y1, y2),
                   query(rc, mid+1, ex,
     x1, x2, y1, y2));
  }
};
```

1.8 Segment Tree 2D (Index compression)

```
// USAGE
ST2D seg(n);
// Phase 1: register all (x, y) points
     that will ever be updated
for (auto& [x, y, v] : updates)
    seg.collect(1, 1, n, x, y);
// Phase 2: compress and allocate inner
     trees
seg.build(1, 1, n);
// Updates and queries
seg.upd   (1, 1, n, x, y, v);
seg.query (1, 1, n, x1, x2, y1, y2);
//
     ////////////////////////////////////

void compress(vector<int>& a) {
    sort(all(a));
    a.erase(unique(all(a)), a.end());
}
int ind(vi& a, int val) {
    return lower_bound(all(a), val) - a.
     begin();
}
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  int n;
  vector<vi> ys;       // compressed y-
     coords per x-node
  vector<vi> t;        // inner seg tree
     per x-node
  ST2D(int n) : n(n), ys(4 * n), t(4 * n
     ) {}
  void collect(int nd, int b, int e, int
     x, int y) {
    ys[nd].pb(y);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) collect(lc, b, mid, x,
      y);
    else          collect(rc, mid + 1, e
     , x, y);
  }
  void build(int nd, int b, int e) {
    compress(ys[nd]);
    t[nd].assign(ys[nd].sz() * 2, 0);
    if (b == e) return;
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
  }
  inline long long combine(long long a,
     long long b) { return max(a, b); }
  void upd_y(int nd, int y, long long v)
     {
    int m = ys[nd].sz();
    int i = ind(ys[nd], y) + m;
    t[nd][i] = combine(t[nd][i], v);
    for (i >>= 1; i >= 1; i >>= 1)
      t[nd][i] = combine(t[nd][i*2], t[
     nd][i*2+1]);
  }
  long long query_y(int nd, int y1, int
     y2) {
    if (y1 > y2) return 0;
    int m = ys[nd].sz();
    int l = ind(ys[nd], y1) + m;
    int r = (int)(upper_bound(all(ys[nd
     ]), y2) - ys[nd].begin() - 1) + m;
    ll mx = 0;
    while (l <= r) {
      if (l%2 == 1) mx = combine(mx, t
     [nd][l++]);
      if (r%2 == 0) mx = combine(mx, t
     [nd][r--]);
      l >>= 1; r >>= 1;
    }
    return mx;
  }
  void upd(int nd, int b, int e, int x,
     int y, long long v) {
    upd_y(nd, y, v);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) upd(lc, b, mid, x, y,
     v);
    else          upd(rc, mid + 1, e, x,
      y, v);
  }
  long long query(int nd, int b, int e,
     int x1, int x2, int y1, int y2) {
    if (x1 > x2 || y1 > y2) return 0;
     // empty range (e.g. a[i].f == 1)
    if (x1 > e  || b > x2)  return 0;
    if (x1 <= b && e <= x2)  return
     query_y(nd, y1, y2);
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, x1,
      x2, y1, y2),
                   query(rc, mid+1, e,
     x1, x2, y1, y2));
} // }
};
```

1.9 Merse Sort Tree with Update

```
// 0 based indexing
template <typename T>
struct MST {
  int n;
  vector<ordered_set<pair<T, int>>> tree
     ;
  vector<T> a;
  MST(vector<T>& v) : a(v) {
    n = v.size();
    tree.resize(4 * n);
    build(0, 0, n - 1, a);
  }
  void merge(int u, int l, int r) {
    tree[u] = tree[l];
    for (auto x : tree[r]) tree[u].
     insert(x);
  }
  void build(int u, int l, int r, vector
     <T>& a) {
    if (l == r) {
      tree[u].insert({a[l], l});
      return;
    }
    int mid = (l + r) / 2;
    build(2 * u + 1, l, mid, a);
    build(2 * u + 2, mid + 1, r, a);
    merge(u, 2 * u + 1, 2 * u + 2);
  }
  // Returns number of elements smaller
     than x in range l, r
  int query(int l, int r, T x) { return
     get(0, 0, n - 1, l, r, x); }
  void update(int i, T x) {
    _update(0, 0, n - 1, i, x);
    a[i] = x;
  }
  int get(int u, int tl, int tr, int l,
     int r, T x) {
    if (tl >= l && tr <= r) return tree[
     u].order_of_key({x, 0});
    if (tl > r || tr < l) return 0;
    int mid = (tl + tr) / 2;
    return get(2 * u + 1, tl, mid, l, r,
      x) +
           get(2 * u + 2, mid + 1, tr, l
     , r, x);
  }
  void _update(int u, int l, int r, int
     i, T x) {
    tree[u].erase({a[i], i});
    tree[u].insert({x, i});
    if (l == r) return;
    int mid = (l + r) / 2;
    if (i <= mid)
      _update(2 * u + 1, l, mid, i, x);
    else
      _update(2 * u + 2, mid + 1, r, i,
     x);
  }
};
```

1.10 Treap

```
const int id = 0, lazy_id = 0;
struct Node {
    int val, sum, lazy = 0;
    int pri, sz = 1;
    Node *l, *r;
    Node(int v, int p) : val(v), sum(v),
      pri(p) { l = r = nullptr; }
};
mt19937 rnd{(uint32_t)chrono::
     steady_clock::now().
     time_since_epoch().count()};
int sz(Node* n) { return n ? n->sz : 0;
     }
int sum(Node* n) { return n ? n->sum :
     id; }
int combine(int x, int y) { return x + y
     ; }
void apply(Node* n, int v) { // change
     here
    if (!n) return;
    n->val += v; n->lazy += v; n->sum +=
      v * n->sz;
}
void push(Node* n) {
    if (n->lazy != lazy_id) { apply(n->l
     , n->lazy); apply(n->r, n->lazy); n
     ->lazy = 0; }
}
void pull(Node* n) {
    n->sz = sz(n->l) + sz(n->r) + 1;
    n->sum = combine(sum(n->l), sum(n->r
     )) + n->val;
}
pair<Node*, Node*> split(Node* n, int k)
     { // splits into 1..k | k+1..n
    if (!n) return {nullptr, nullptr};
    push(n);
    int sl = sz(n->l);  // n->val for
     key
    if (sl < k) {
      auto [a, b] = split(n->r, k - sl -
     1);
      n->r = a; pull(n); return {n, b};
    } // else for split_by_key
    auto [a, b] = split(n->l, k);
    n->l = b; pull(n); return {a, n};
}
Node* merge(Node* a, Node* b) {
    if (!a) return b;
    if (!b) return a;
```

```
    if (a->pri > b->pri) { push(a); a->r
      = merge(a->r, b); pull(a); return
     a; }
    push(b); b->l = merge(a, b->l); pull
     (b); return b;
}
// Merges two OVERLAPPING sorted treaps
     O(K log(N/K))
Node* meld(Node* u, Node* v) {
    if (!u || !v) return u ? u : v;
    if (u->pri < v->pri) swap(u, v);
    auto [l, r] = split_by_key(v, u->val
     );
    u->l = meld(u->l, l);
    u->r = meld(u->r, r);
    pull(u);
    return u;
}
void clear(Node* n) {
    if (!n) return;
    clear(n->l); clear(n->r); delete n;
}
Node* insert(Node* root, int val, int
     pos = 0) {
    Node* n = new Node(val, rnd());
    pos = pos ? pos - 1 : sz(root);
    auto [a, b] = split(root, pos);
    root = merge(merge(a, n), b);
    return root;
}
Node* erase(Node* root, int pos) { // 1-
     based
    auto [a, b] = split(root, pos);
    auto [c, d] = split(a, pos - 1);
    root = merge(c, b); if (d) delete d;
    return root;
}
int get(int i, Node* n) { // 1-based
     index
    if(sz(n) < i) return -1;
    push(n);
    int id = sz(n->l) + 1;  //
    if (id == i) return n->val;
    return id < i ? get(i - id, n->r) :
     get(i, n->l);
}
int search(int val, Node* n) {  // in
     sorted
    if(!n) return 0;
    push(n);
    int id = sz(n->l) + 1;
    int v = n->val;
    // cout << id << ' ' << v << '\n';
    if(v < val) return id+search(val, n
     ->r);
    else return search(val, n->l);
}
```

1.11 Persistent DSU

```
struct DSU {
  vector<vector<pair<int, int>>> par;
  int time = 0; //initial time
  DSU(int n) : par(n + 1, {{-1, 0}}) {}
  bool merge(int u, int v) {
    ++time;
    if ((u = root(u, time)) == (v = root
     (v, time))) return 0;
    if (par[u].back().first > par[v].
     back().first) swap(u, v);
    par[u].push_back({par[u].back().
     first + par[v].back().first, time})
     ;
    par[v].push_back({u, time}); //par[v
     ] = u
    return 1;
  }
  bool same(int u, int v, int t) {
    return root(u, t) == root(v, t);
  }
  int root(int u, int t) { //root of u
     at time t
    if (par[u].back().first >= 0 && par[
     u].back().second <= t) return root(
     par[u].back().first, t);
    return u;
  }
  int size(int u, int t) { //size of the
       component of u at time t
    u = root(u, t);
    int l = 0, r = (int) par[u].size() -
     1, ans = 0;
    while (l <= r) {
      int mid = l + r >> 1;
      if (par[u][mid].second <= t) ans =
     mid, l = mid + 1;
      else r = mid - 1;
    }
    return -par[u][ans].first;
  }
};
```

1.12 MOs Algorithm

```
void add(int ind, int end, int L, int R)
     { /* ... */ }
void del(int ind, int end, int L, int R)
     { /* ... */ }
int calc(int L, int R) { /* ... */ }
vector<int> mo(vector<pair<int, int>>& Q
     ) {
  int L = 1, R = 0, blk = 350;
  vector<int> s(Q.size()), res(Q.size())
     ;
  iota(s.begin(), s.end(), 0);
  sort(s.begin(), s.end(), [&](int i,
     int j) {
    int bi = Q[i].first / blk;
    int bj = Q[j].first / blk;
    if (bi != bj) return bi < bj;
    if (bi & 1) return Q[i].second < Q[j
    ].second;
    return Q[i].second > Q[j].second;
  });
  for (int qi : s) {
    pair<int, int> q = Q[qi];
    int ql = q.first;
    int qr = q.second;
    while (L > ql) { --L; add(L, 0, L, R
     ); }
    while (R < qr) { ++R; add(R, 1, L, R
     ); }
    while (L < ql) { del(L, 0, L, R); L
     ++; }
    while (R > qr) { del(R, 1, L, R); R
     --; }
    res[qi] = calc(L, R);
  }
  return res;
}
```

1.13 MOs with updates

```
struct mos_update {
  ll a[100005];
  ll ans;
  mos_update() { /* ... */ }
  void add(int ind, int end, int L, int
     R) { /* ... */ }
  void del(int ind, int end, int L, int
     R) { /* ... */ }
  void apply(int pos, int val, int L,
     int R) {
    if (L <= pos && pos <= R) {
      del(pos, -1, L, R); a[pos] = val;
     add(pos, -1, L, R);
    } else {
      a[pos] = val;
    }
  }
  vector<ll> mo(vector<vector<int>>& Q,
     vector<vector<int>>& U, int n) {
    int L = 1, R = 0, T = 0;
    ans = 0;
    int blk = max(1, (int)pow(Q.size(),
     0.666));
    vector<int> s(Q.size());
    vector<ll> res(Q.size());
    iota(s.begin(), s.end(), 0);
    sort(s.begin(), s.end(), [&](int i,
     int j) {
      int li = Q[i][0] / blk, lj = Q[j
     ][0] / blk;
      if (li != lj) return li < lj;
      int ri = Q[i][1] / blk, rj = Q[j
     ][1] / blk;
      if (ri != rj) return (li & 1) ? ri
     < rj : ri > rj;
      return (ri & 1) ? Q[i][2] < Q[j
     ][2] : Q[i][2] > Q[j][2];
    });
    for (int qi : s) {
      int ql = Q[qi][0], qr = Q[qi][1],
     qt = Q[qi][2];
      while (T < qt) { apply(U[T][0], U[
     T][2], L, R); T++; }
      while (T > qt) { --T; apply(U[T
     ][0], U[T][1], L, R); }
      while (L > ql) { --L; add(L, 0, L,
     R); }
      while (R < qr) { ++R; add(R, 1, L,
     R); }
      while (L < ql) { del(L, 0, L, R);
     L++; }
      while (R > qr) { del(R, 1, L, R);
     R--; }
      res[qi] = ans;
    }
    return res;
  }
};
```

1.14 MOs on Trees

```
// to be added
```

1.15 Dynamic Connectivity

```
struct ST {
#define lc (n << 1)
#define rc ((n << 1) | 1)
  vector<query> t[maxN*4];
  ST(int n) {}
  void upd(int n, int b, int e, int ql,
     int qr, query& v) {
    if(ql <= b && e <= qr) {
        t[n].push_back(v);
        return;
    }
    int mid = (b + e) >> 1;
    if (ql <= mid) upd(lc, b, mid, ql,
     qr, v);
    if(qr > mid) upd(rc, mid + 1, e, ql,
      qr, v);
  }
  // can be generalized to any rollback
     structure
  void dfs(int n, int b, int e, int root
     , int eg) {
    int sz = t[n].size(), init = pst.T,
     croot = root;
    for(auto u : t[n]) croot = pst.upd(
     croot, 1, eg, u.a, u.b, u.h);
    if(b == e) {  // query ans
        if(!(ask[b][0] == 0 && ask[b][1]
      == 0))
            ans[b] = pst.query(croot, 1,
     eg, ask[b][0], ask[b][1]);
    } else {
        int mid = (b+e) >> 1;
        dfs(lc, b, mid, croot, eg); dfs(
     rc, mid+1, e, croot, eg);
    }
    pst.T = init; // rollback
  }
  #undef lc
  #undef rc
};
```

1.16 Xor Basis

```
const int LOG_K = 60;  // Adjust as
     needed
struct XorBasis {
  vector<int64_t> basis;
  int64_t N = 0, tmp = 0;
  void add(int64_t x) {
    N++; tmp |= x;
    for (auto& i : basis) x = min(x, x ^
      i);
    if (!x) return;
    for (auto& i : basis) if ((i ^ x) <
     i) i ^= x;
    basis.push_back(x);
    sort(basis.begin(), basis.end());
  }
```

```
  int64_t size() { return basis.size();
     }
  void clear() { N = 0; tmp = 0; basis.
     clear(); }
  bool minxor(int64_t x) {
    for (auto& i : basis) x = min(x, x ^
      i);  // for maxxor, return max
    return !x;  // for possible(x),
     return x;
  }
  int64_t cntxor(int64_t x) {
    if (!possible(x)) return 0;
    return 1LL << (N - size());
  }
  int64_t sumOfAll() { return tmp * (1LL
     << (N - 1)); }
  int64_t kth(int64_t k) {
    int64_t sz = size();
    if (k > (1LL << sz)) return -1;
    k--;
    int64_t ans = 0;
    for (int64_t i = 0; i < sz; i++) if
     (k >> i & 1) ans ^= basis[i];
    return ans;
  }
};
```

1.17 Link Cut Tree

```
struct node {
  int p = 0, c[2] = {0, 0}, sz = 0, ssz
     = 0, vsz = 0, mn_id = 0;
  bool flip = false;
  long long val = INF, sum = 0, lazy =
     0, subsum = 0, vsum = 0, mn = INF;
  node() {}
  node(int id, long long x) : val(x),
     sum(x), subsum(x), mn(x), mn_id(id)
     , sz(1), ssz(1) {}
};
struct LCT {
  vector<node> t;
  LCT(int n, int m = 0) : t(n + m + 1) {
    for (int i = 1; i <= n + m; i++) t[i
     ] = node(i, INF);
    t[0] = node();
    t[0].sz = t[0].ssz = t[0].vsz = t
     [0].sum = t[0].subsum = t[0].vsum =
      0;
  }
  bool is_root(int x) { return t[t[x].p
     ].c[0] != x && t[t[x].p].c[1] != x;
      }
  int dir(int x) { return t[t[x].p].c[1]
      == x; }
  void flip(int x) {
    if (x) swap(t[x].c[0], t[x].c[1]), t
     [x].flip ^= 1;
  }
  void add(int x, long long v) {
    if (!x) return;
    t[x].val += v; t[x].mn += v; t[x].
     lazy += v;
    t[x].sum += v * t[x].sz; t[x].subsum
      += v * t[x].ssz; t[x].vsum += v *
     t[x].vsz;
  }
  void push(int x) {
    if (!x) return;
    if (t[x].flip) flip(t[x].c[0]), flip
     (t[x].c[1]), t[x].flip = false;
    if (t[x].lazy) add(t[x].c[0], t[x].
     lazy), add(t[x].c[1], t[x].lazy), t
     [x].lazy = 0;
  }
  void pull(int x) {
    if (!x) return;
    int l = t[x].c[0], r = t[x].c[1];
    t[x].sz = t[l].sz + t[r].sz + 1;
    t[x].ssz = t[l].ssz + t[r].ssz + t[x
     ].vsz + 1;
    t[x].sum = t[l].sum + t[r].sum + t[x
     ].val;
    t[x].subsum = t[l].subsum + t[r].
     subsum + t[x].vsum + t[x].val;
    t[x].mn = t[x].val; t[x].mn_id = x;
    if (t[l].mn < t[x].mn) t[x].mn = t[l
     ].mn, t[x].mn_id = t[l].mn_id;
    if (t[r].mn < t[x].mn) t[x].mn = t[r
     ].mn, t[x].mn_id = t[r].mn_id;
  }
  void push_all(int x) {
    if (!is_root(x)) push_all(t[x].p);
    push(x);
  }
  void rotate(int x) {
    int y = t[x].p, z = t[y].p, dx = dir
     (x), dy = dir(y), w = t[x].c[!dx];
    if (!is_root(y)) t[z].c[dy] = x;
    t[x].p = z; t[x].c[!dx] = y; t[y].p
     = x; t[y].c[dx] = w;
    if (w) t[w].p = y;
    pull(y); pull(x);
  }
  void splay(int x) {
    push_all(x);
    while (!is_root(x)) {
      int y = t[x].p;
      if (!is_root(y)) rotate(dir(x) ==
     dir(y) ? y : x);
      rotate(x);
    }
  }
  int access(int u) {
    int last = 0;
    for (int v = 0; u; v = u, u = t[u].p
     ) {
      splay(u);
      int r = t[u].c[1];
      t[u].vsz += t[r].ssz - t[v].ssz;
      t[u].vsum += t[r].subsum - t[v].
     subsum;
      t[u].c[1] = v; pull(u); last = u;
    }
    return last;
  }
  void make_root(int u) { access(u);
     splay(u); flip(u); }
  int find_root(int u) {
    access(u); splay(u); push(u);
    while (t[u].c[0]) u = t[u].c[0],
     push(u);
    splay(u);
    return u;
  }
  bool connected(int u, int v) { return
     find_root(u) == find_root(v); }
  void link(int u, int v) {
    make_root(u);
    if (find_root(v) == u) return;
    access(v); splay(v);
    t[u].p = v; t[v].vsz += t[u].ssz; t[
     v].vsum += t[u].subsum; pull(v);
  }
  void cut(int u, int v) {
    make_root(u); access(v); splay(v);
    if (t[v].c[0] == u && !t[u].c[1]) t[
     v].c[0] = t[u].p = 0, pull(v);
  }
  // --- Operations ---
  void link_edge(int e, int u, int v,
     long long w) { set_val(e, w); link(
     u, e); link(e, v); }
  void cut_edge(int e, int u, int v) {
     cut(u, e); cut(e, v); }
  void set_val(int u, long long x) {
     access(u); splay(u); t[u].val = x;
     pull(u); }
  void add_val(int u, long long x) {
     access(u); splay(u); t[u].val += x;
      pull(u); }
  void add_path(int u, int v, long long
     x) { make_root(u); access(v); splay
     (v); add(v, x); }
  pair<long long, int> query_min(int u,
     int v) { make_root(u); access(v);
     splay(v); return {t[v].mn, t[v].
     mn_id}; }
  long long query_path_sum(int u, int v)
      { make_root(u); access(v); splay(v
     ); return t[v].sum; }
  long long query_subtree_sum(int u, int
      root = 1) { make_root(root);
     access(u); splay(u); return t[u].
     val + t[u].vsum; }
};
```

1.18 Wavelet Tree

```
struct wavelet_tree {
  int lo, hi;
  wavelet_tree *l = nullptr, *r =
     nullptr;
  vector<int> b, c;
  wavelet_tree() : lo(0), hi(0) {}
  void init(int *from, int *to, int x,
     int y) {
    lo = x; hi = y;
    if (from >= to) return;
    int mid = lo + (hi - lo) / 2;
    auto f = [mid](int val) { return val
      <= mid; };
    b.reserve(to - from + 1); c.reserve(
     to - from + 1);
    b.push_back(0); c.push_back(0);
    for (auto it = from; it != to; ++it)
      {
      b.push_back(b.back() + f(*it));
      c.push_back(c.back() + *it);
    }
    if (lo == hi) return;
    auto pivot = stable_partition(from,
     to, f);
    (l = new wavelet_tree())->init(from,
      pivot, lo, mid);
    (r = new wavelet_tree())->init(pivot
     , to, mid + 1, hi);
  }
  int kth(int l, int r, int k) {
    if (l > r) return 0;
    if (lo == hi) return lo;
    int inLeft = b[r] - b[l - 1], lb = b
     [l - 1], rb = b[r];
    if (k <= inLeft) return this->l->kth
     (lb + 1, rb, k);
    return this->r->kth(l - lb, r - rb,
     k - inLeft);
  }
  int LTE(int l, int r, int k) {
    if (l > r || k < lo) return 0;
    if (hi <= k) return r - l + 1;
    int lb = b[l - 1], rb = b[r];
    return l->LTE(lb + 1, rb, k) + r->
     LTE(l - lb, r - rb, k);
  }
  int count(int l, int r, int k) {
    if (l > r || k < lo || k > hi)
     return 0;
    if (lo == hi) return r - l + 1;
    int lb = b[l - 1], rb = b[r], mid =
     lo + (hi - lo) / 2;
    if (k <= mid) return l->count(lb +
     1, rb, k);
    return r->count(l - lb, r - rb, k);
  }
  int sum(int l, int r, int k) {
    if (l > r || k < lo) return 0;
    if (hi <= k) return c[r] - c[l - 1];
    int lb = b[l - 1], rb = b[r];
    return l->sum(lb + 1, rb, k) + r->
     sum(l - lb, r - rb, k);
  }
  ~wavelet_tree() { delete l; delete r;
     }
};
```

1.19 Sparse Table (RMQ)

```
template<typename T>
struct SparseTable {
  const int N = 1e5 + 9;
  vector<vector<T>> t;
  vector<T> a;
  T combine(T x, T y) { return min(x, y)
     ; }
  void build(int n) {
    t.assign(n + 1, vector<T>(18));
    for(int i = 1; i <= n; ++i) t[i][0]
     = a[i];
    for(int k = 1; k < 18; ++k)
      for(int i = 1; i + (1 << k) - 1 <=
     n; ++i)
        t[i][k] = combine(t[i][k-1], t[i
     + (1 << (k-1))][k-1]);
  }
  T query(int l, int r) {
```

```
    int k = 31 - __builtin_clz(r - l +
     1);
    return combine(t[l][k], t[r - (1 <<
     k) + 1][k]);
  }
};
```

2 Geometry

2.1 Geometry Primitives

```
#define EPS 1e-12
#define pi 3.14159265358979311599796346\
85441851615905761718751
bool feq(const long double& a, const
     long double& b) {
  long double del = fabsl(a - b);
  if (del <= EPS) return true;
  return del <= EPS * max(fabsl(a),
     fabsl(b));
}
```

2.2 Point 2D

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T>
int sgn(T x) {
  return (x > 0) - (x < 0);
  // return (x > EPS) - (x < -EPS); //
     for float
}
template <class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x
     ), y(y) {}
  bool operator<(P p) const { return tie
     (x, y) < tie(p.x, p.y); }
  bool operator==(P p) const { return
     tie(x, y) == tie(p.x, p.y); } //
     Use feq for float
  P operator+(P p) const { return P(x +
     p.x, y + p.y); }
  P operator-(P p) const { return P(x -
     p.x, y - p.y); }
  P operator*(T d) const { return P(x *
     d, y * d); }
  P operator/(T d) const { return P(x /
     d, y / d); }
  T dot(P p) const { return x * p.x + y
     * p.y; }
  T cross(P p) const { return x * p.y -
     y * p.x; }
  T cross(P a, P b) const { return (a -
     *this).cross(b - *this); }
  T dist2() const { return x * x + y * y
     ; }
  T dist2(P p) const { return (*this - p
     ).dist2(); }
  long double dist() const { return sqrt
     ((long double)dist2()); }
  long double dist(P p) const { return
     (*this - p).dist(); }
  // angle to x-axis in interval [-pi,
     pi]
  double angle() const { return atan2(y,
      x); }
  long double angle(P p) const {
    return atan2((long double)this->
     cross(p), (long double)this->dot(p)
     );
  }
  P unit() const { return *this / dist()
     ; }  // makes dist()=1
  P perp() const { return P(-y, x); }
         // rotates +90 degrees
  P normal() const { return perp().unit
     (); }
  // returns point rotated 'a' radians
     ccw around the origin
  P rotate(long double a) const {
    return P(x * cos(a) - y * sin(a), x
     * sin(a) + y * cos(a));
  }
  P project(P p) const { return *this *
     ((double)dot(p) / dist2()); }
  double scalar_project(P p) const {
     return (double)dot(p) / dist(); }
  P reflect(P p) const { return project(
     p) * 2 - p; }
  // Orientation test: >0 for left turn,
      <0 for right turn, 0 if collinear
  T side(P p1, P p2) const { return (p1
     - *this).cross(p2 - *this); }
  friend ostream& operator<<(ostream& os
     , P p) {
    return os << "(" << p.x << "," << p.
     y << ")";
  }
  // Same direction, length r (returns *
     this if degenerate)
  P truncate(double r) const { double k
     = dist(); return !sgn(k) ? *this :
     *this * (r / k); }
};
```

2.3 Polar Sort

```
bool half(P p) {
  return p.y < 0 || (p.y == 0 && p.x <
     0);
}
void polar_sort(vector<P>& v, P o = P(0,
      0)) {
  sort(v.begin(), v.end(), [&](P a, P b)
     {
    P u = a - o, w = b - o;
    return make_tuple(half(u), 0.0, u.
     dist2()) < make_tuple(half(w), u.
     cross(w), w.dist2());
  });
}
```

2.4 Line Intersection

Description:
Usage:
```
 auto res = lineInter(s1,e1,s2,e2); if
(res.first == 1) cout << "intersection point
at " << res.second << endl;
```

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2,
      P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0)  // if parallel
    return {-(s1.cross(e1, s2) == 0), P
     (0, 0)};
  auto p = s2.cross(e1, e2), q = s2.
     cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

2.5 Line Equation

```
array<T, 3> line_equation(const Point<T
     >&b, const Point<T>&c){
 array<T, 3>res;
 res[0] = b.y - c.y, res[1] = -(b.x - c.
     x), res[2] = b.x*c.y - b.y*c.x;
 if(res[0] < 0)res[0] = -res[0], res[1]
     = -res[1], res[2] = -res[2];
 else if(res[0] == 0 and res[1] < 0)res
     [1] = -res[1], res[2] = -res[2]; //
      for double use feq(res[0], 0)
 return res;
}
```

2.6 Segment Distance

Description:
Usage:
```
 Point<double> a, b(2,2), p(1,1); bool
onSegment = segDist(a,b,p) < 1e-10;
```

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s == e) return (p - s).dist();
  auto d = (e - s).dist2(), t = min(d,
     max(.0, (p - s).dot(e - s)));
  return ((p - s) * d - (e - s) * t).
     dist() / d;
}
```

2.7 Segment Intersection

Description:
Usage:
```
vector<P> inter = segInter(s1,e1,s2,e2); if
(sz(inter)==1) cout << "segments intersect at
" << inter[0] << endl;
```

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(
     d, b), oc = a.cross(b, c),
       od = a.cross(b, d);
  // Checks if intersection is single
     non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) *
     sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa
     )};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

2.8 Side Of

Description:
Usage:
```
bool left = sideOf(p1,p2,q)==1;
```

```
template <class P>
int sideOf(P& s, P& e, P& p) {
  return sgn(s.cross(e, p));
}
template <class P>
int sideOf(const P& s, const P& e, const
      P& p, double eps) {
  auto a = (e - s).cross(p - s);
  double l = (e - s).dist() * eps;
  return (a > l) - (a < -l);
}
```

2.9 On Segment

Description: Returns true iff p lies on the line segment from s to e.
Usage:
```
\texttt {(segDist(s,e,p)<=epsilon)} instead
when using Point<double>.
```

```
template <class P>
bool onSegment(P s, P e, P p) {
  double d = (e - s).dist();
  if (d < 1e-9) return (p - s).dist() <=
     1e-9;
  return abs((e - s).cross(p - s)) / d
     <= 1e-9 && (s - p).dot(e - p) <= 1e
     -9;
}
```

2.10 Linear Transformation

Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
typedef Point<double> P;
P linearTransformation(const P& p0,
     const P& p1, const P& q0, const P&
     q1,
                       const P& r) {
  P dp = p1 - p0, dq = q1 - q0, num(dp.
     cross(dq), dp.dot(dq));
  return q0 + P((r - p0).cross(num), (r
     - p0).dot(num)) / dp.dist2();
}
```

2.11 Circle Intersection

```
typedef Point<double> P;
bool circleInter(P a, P b, double r1,
     double r2, pair<P, P>* out) {
  if (a == b) {
    assert(r1 != r2);
    return false;
  }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1 + r2
     , dif = r1 - r2,
```

```
        p = (d2 + r1 * r1 - r2 * r2) /
    (d2 * 2), h2 = r1 * r1 - p * p * d2
    ;
  if (sum * sum < d2 || dif * dif > d2)
    return false;
  P mid = a + vec * p, per = vec.perp()
    * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

2.12 Circle Polygon Intersection

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.
    dot(q))
double circlePoly(P c, double r, vector<
    P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p) / d.dist2(), b = (
     p.dist2() - r * r) / d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a - sqrt(det)), t
     = min(1., -a + sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q
    ) * r2;
    P u = p + d * s, v = q + d * (t - 1)
    ;
    return arg(p, u) * r2 + u.cross(v) /
     2 + arg(v, q) * r2;
  };
  auto sum = 0.0;
  rep(i, 0, sz(ps)) sum += tri(ps[i] - c
    , ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```

2.13 Circle Line Intersection

```
template <class P>
vector<P> circleLine(P c, double r, P a,
     P b) {
  P ab = b - a, p = a + ab * (c - a).dot
    (ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r * r -
     s * s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

2.14 Circle Segment Intersection

```
template <class P>
vector<P> circleSegmentIntersection(P c,
     double r, P a, P b) {
  P ab = b - a;
  double l2 = ab.dist2();
  if (l2 < 1e-12) return (a - c).dist()
    <= r + 1e-9 ? vector<P>{a} : vector
    <P>{};
  P p = a + ab * (c - a).dot(ab) / l2;
  double h2 = r * r - pow((c - a).cross(
    ab), 2) / l2;
  if (h2 < -1e-9) return {};
  P h = ab.unit() * sqrt(max(0.0, h2));
  vector<P> lpts = abs(h2) < 1e-9 ?
    vector<P>{p} : vector<P>{p - h, p +
     h}, res;
  for (auto &pt : lpts) {
    if ((a - pt).dot(b - pt) <= 1e-9)
    res.push_back(pt);
  }
  if (res.empty() && (a - c).dist() <= r
     + 1e-9 && (b - c).dist() <= r + 1e
    -9) {
    res = {a, b};
  }
  return res;
}
```

2.15 Circle Tangents

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. $O(n)$

```
template <class P>
vector<pair<P, P>> tangents(P c1, double
     r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(),
    h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0) return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2)
    * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v *
     r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

2.16 Minimum Enclosing Circle

Description: Computes the minimum circle that encloses a set of points. $O(n)$

```
bool inside(const P& p, const P& c,
    double r) {
  return (p - c).dist() <= r + 1e-9;
}
// For generalized, udpate calc with
    gaussian elimination
pair<double, P> calc(const vector<P>& R)
     {
  if (R.empty()) return {0, P()};
  if (R.size() == 1) return {0, R[0]};
  P c = R.size() == 2 ? (R[0] + R[1]) /
    2 : ccCenter(R[0], R[1], R[2]);
  return {(c - R[0]).dist(), c};
}
pair<double, P> welzl(vector<P>& p,
    vector<P>& R, int n, int D) {
  if (!n || R.size() == D + 1) return
    calc(R);
  auto res = welzl(p, R, n - 1, D);
  if (!inside(p[n - 1], res.second, res.
    first)) {
    R.push_back(p[n - 1]);
    res = welzl(p, R, n - 1, D);
    R.pop_back();
  }
  return res;
}
pair<double, P> mec(vector<P>& p, int D
    = 2) {
  shuffle(p.begin(), p.end(), mt19937
    (1337));
  vector<P> R;
  return welzl(p, R, p.size(), D);
}
```

2.17 Inside Polygon

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow. $O(n)$

Usage:

```
vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool
in = inPolygon(v, P{3, 3}, false);
```

```
bool is_point_on_polygon(vector<P> &p, P
    & z) {
  int n = p.size();
  for (int i = 0; i < n; i++) {
    if (onSegment(p[i], p[(i + 1) % n
    ], z)) return 1;
  }
  return 0;
}
int winding_number(vector<P>& p, P& z) {
    // O(n)
  if (is_point_on_polygon(p, z)) return
    1e9;
  int n = p.size(), ans = 0;
  for (int i = 0; i < n; ++i) {
    int j = (i + 1) % n;
    bool below = p[i].y < z.y;
    if (below != (p[j].y < z.y)) {
      auto orient = z.cross(p[j], p[i]);
      if (abs(orient) < 1e-9) return 0;
      if (below == (orient > 0)) ans +=
     below ? 1 : -1;
    }
  }
  return ans;
}
// -1 if strictly inside, 0 if on the
    polygon, 1 if strictly outside
int is_point_in_polygon(vector<P>& p,  P
    & z) { // O(n)
  int k = winding_number(p, z);
  return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}
```

2.18 Polygon Area

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
template <class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  for(int i = 0; i < (int) v.size() - 1;
     ++i)
    a += v[i].cross(v[i + 1]);
  return a;
}
```

2.19 Polygon Center

Description: Returns the center of mass for a polygon. Time: $O(n)$

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0);
  double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(
    v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].
     cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

2.20 Polygon Cut

```
vector<P> polygonCut(const vector<P>&
    poly, P s, P e) {
  vector<P> res;
  for (int i = 0; i < (int)poly.size();
     ++i) {
    P cur = poly[i], prev = i ? poly[i -
     1] : poly.back();
    auto a = s.cross(e, cur), b = s.
     cross(e, prev);
    if ((a < 0) != (b < 0)) res.
     push_back(cur + (prev - cur) * (a /
     (a - b)));
    if (a < 0) res.push_back(cur);
  }
  return res;
}
```

2.21 Polygon Union

```
double rat(P a, P b) { return sgn(b.x) ?
     a.x / b.x : a.y / b.y; }
double polyUnion(vector<vector<P>>& poly
    ) {
  double ret = 0;
  for (int i = 0; i < (int)poly.size();
    ++i) for (int v = 0; v < (int)poly[
    i].size(); ++v) {
    P A = poly[i][v], B = poly[i][(v +
     1) % (int)poly[i].size()];
    vector<pair<double, int>> segs =
     {{0, 0}, {1, 0}};
    for (int j = 0; j < (int)poly.size()
     ; ++j) if (i != j) {
```

```
    for (int u = 0; u < (int)poly[j].
    size(); ++u) {
      P C = poly[j][u], D = poly[j][(u
     + 1) % (int)poly[j].size()];
      int sc = sideOf(A, B, C), sd =
    sideOf(A, B, D);
      if (sc != sd) {
        double sa = C.cross(D, A), sb
    = C.cross(D, B);
        if (min(sc, sd) < 0) segs.
    emplace_back(sa / (sa - sb), sgn(sc
     - sd));
      } else if (!sc && !sd && j < i
    && sgn((B - A).dot(D - C)) > 0) {
        segs.emplace_back(rat(C - A, B
     - A), 1);
        segs.emplace_back(rat(D - A, B
     - A), -1);
      }
    }
    sort(segs.begin(), segs.end());
    for (auto& s : segs) s.first = min(
     max(s.first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    for (int j = 1; j < (int)segs.size()
     ; ++j) {
      if (!cnt) sum += segs[j].first -
     segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```

2.22 Extreme Vertex

```
// id of the vertex having maximum dot
    product with z
// polygon must be convex
// top - upper right vertex
// for minimum dot product negate z and
    return -p[id].dot(z)
int extreme_vertex(const vector<P>& p,
    const P& z, const int top) { // O(
    log n)
  int n = p.size();
  if (n == 1) return 0;
  auto ans = p[0].dot(z);
  int id = 0;
  if (p[top].dot(z) > ans) ans = p[top].
    dot(z), id = top;
  int l = 1, r = top - 1;
  while (l < r) {
    int mid = (l + r) / 2;
    if (p[mid + 1].dot(z) >= p[mid].dot(
    z)) l = mid + 1;
    else r = mid;
  }
  if (p[l].dot(z) > ans) ans = p[l].dot(
    z), id = l;
  l = top + 1, r = n - 1;
  while (l < r) {
    int mid = (l + r) / 2;
    if (p[(mid + 1) % n].dot(z) >= p[mid
    ].dot(z)) l = mid + 1;
    else r = mid;
  }
  l %= n;
  if (p[l].dot(z) > ans) ans = p[l].dot(
    z), id = l;
  return id;
}
```

2.23 Manhattan MST

```
vector<array<int, 3>> manhattanMST(
    vector<P> ps) {
  vector<int> id(ps.size());
  iota(id.begin(), id.end(), 0);
  vector<array<int, 3>> edges;
  for (int k = 0; k < 4; ++k) {
    sort(id.begin(), id.end(),
        [&](int i, int j) { return (ps[
    i] - ps[j]).x < (ps[j] - ps[i]).y;
    });
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-
    ps[i].y); it != sweep.end();
          sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j
    });
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps)
      if (k & 1)
        p.x = -p.x;
      else
        swap(p.x, p.y);
  }
  return edges;
}
```

2.24 Convex Hull

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. $O(nlogn)$

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  int n = pts.size();
  if (n <= 1) return pts;
  sort(all(pts));
  vector<P> h(n + 1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t,
     reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t - 2].
    cross(h[t - 1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t
    == 2 && h[0] == h[1])};
}
```

2.25 Hull Diameter

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). $O(n)$

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S
    [0]}});
  rep(i, 0, j) for (;; j = (j + 1) % n)
    {
    res = max(res, {(S[i] - S[j]).dist2
    (), {S[i], S[j]}});
    if ((S[(j + 1) % n] - S[j]).cross(S[
    i + 1] - S[i]) >= 0) break;
  }
  return res.second;
}
```

2.26 Point Inside Hull

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included. $O(logN)$

```
typedef Point<ll> P;
bool inHull(vector<P>& l, P p, bool
    strict = true) {
  int n = l.size(), a = 1, b = n - 1, r
    = !strict;
  if (n < 3) return r && onSegment(l[0],
     l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap
    (a, b);
  if (sideOf(l[0], l[a], p) >= r ||
    sideOf(l[0], l[b], p) <= -r) return
     false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a)
    = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

2.27 Line Hull Intersection

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:

- $(-1,-1)$ if no collision,
- $(i,-1)$ if touching the corner i,
- (i,i) if along side $(i,i+1)$,
- (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$.

In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line. $O(logN)$

```
#define cmp(i, j) sgn(dir.perp().cross(
    poly[(i) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 &&
    cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m
     + 1, m);
    (ls < ms || (ls == ms && ls == cmp(
    lo, m)) ? hi : lo) = m;
  }
  return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<
    P>& poly) {
  int endA = extrVertex(poly, (a - b).
    perp());
  int endB = extrVertex(poly, (b - a).
    perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i, 0, 2) {
    int lo = endB, hi = endA, n = sz(
    poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 :
     n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi)
    = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0],
    -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly)
    + 1) % sz(poly)) {
      case 0:
        return {res[0], res[0]};
      case 2:
        return {res[1], res[1]};
    }
  return res;
}
```

2.28 Half Plane Intersection

```
typedef Point<double> P;
using Line = pair<P, P>;
bool bad(Line a, Line b, Line c) {
  auto res = lineInter(a.first, a.second
    , b.first, b.second);
  return res.first > 0 && sideOf(c.first
    , c.second, res.second, EPS) < 0;
}
vector<P> halfPlaneIntersection(vector<
    Line>& L) {
  auto ang = [](Line l) { return (l.
    second-l.first).angle(); };
  sort(all(L), [&](Line a,Line b) {
    return ang(a)<ang(b); });
```

```
  vector<Line> lines;
  for (int i = 0; i < L.size(); i++) {
    if (i && abs(ang(L[i])-ang(L[i-1]))<
     EPS) {
      if (sideOf(L[i].first,L[i].second,
     lines.back().first,EPS) < 0) {
        lines.pop_back(); lines.
     push_back(L[i]);
      }
    } else lines.push_back(L[i]);
  }
  int n = lines.size(), qf = 0, qb = 0;
  vector<Line> q(n);
  for (int i = 0; i < n; i++) {
    while (qb-qf >= 2 && bad(q[qb-1], q[
     qb-2], lines[i])) qb--;
    while (qb-qf >= 2 && bad(q[qf], q[qf
     +1], lines[i])) qf++;
    q[qb++] = lines[i];
  }
  while (qb-qf >= 3 && bad(q[qb-1], q[qb
     -2], q[qf])) qb--;
  while (qb-qf >= 3 && bad(q[qf], q[qf
     +1], q[qb-1])) qf++;
  if(qb-qf < 3) return {};
  vector<P> poly;
  for (int i = qf; i < qb; i++) {
    int nxt = (i+1 == qb) ? qf : i + 1;
    poly.push_back(lineInter(q[i].first,
      q[i].second, q[nxt].first, q[nxt].
     second).second);
  }
  return poly;
}
```

2.29 Minkowski Sum Dif

```
using P = Point<ld>;
void reorder(vector<P>& poly) {
  int pos = 0;
  for (int i = 1; i < (int)poly.size();
     ++i)
    if (poly[i] < poly[pos]) pos = i;
  rotate(poly.begin(), poly.begin() +
     pos, poly.end());
}
vector<P> minkowski(vector<P> a, vector<
     P> b) {
  reorder(a), reorder(b);
  int n = a.size(), m = b.size(), ai =
     0, bi = 0;
  a.push_back(a[0]), a.push_back(a[1]);
  b.push_back(b[0]), b.push_back(b[1]);
  vector<P> c;
  while (ai < n || bi < m) {
    c.push_back(a[ai] + b[bi]);
    if (ai < n && bi < m) {
      auto ori = (a[ai + 1] - a[ai]).
     cross(b[bi + 1] - b[bi]);
      if (ori > 1e-9) ai++;
      else if (ori < -1e-9) bi++;
      else ai++, bi++;
    } else if (ai < n) ai++;
    else bi++;
  }
  return c;
}
vector<P> minkowski_dif(vector<P> a,
     vector<P> b) {
  for (auto& p : b) p.x = -p.x, p.y = -p
     .y;
  return minkowski(a, b);
}
```

2.30 Maximum Inscribed Circle

```
ld maxInscribedCircle(vector<P>& poly) {
  ld lo = 0, hi = 1e6, ans = 0, L = 1e9;
  int n = poly.size();
  while (hi - lo >= 1e-12) {
    ld mid = lo + (hi - lo) / 2.0;
    vector<Line> l = {{{-L, -L}, {L, -L
     }}, {{L, -L}, {L, L}}, {{L, L}, {-L
     , L}}, {{-L, L}, {-L, -L}}};
    for (int i = 0; i < n; ++i) {
      auto a = poly[i], b = poly[(i + 1)
     % n];
      auto c = (b - a).unit().perp();
      l.push_back({a + c * mid, b + c *
     mid});
    }
    auto cur = halfPlaneIntersection(l);
    if (!cur.empty()) ans = lo = mid;
    else hi = mid;
  }
  return ans;
}
```

2.31 Rotating Calipers

```
// Outer loop advances edge (i -> i+1),
     inner loop advances antipodal
     pointer j
for (int i = 0, j = 1; i < n; ++i) {
  while ((p[(i + 1) % n] - p[i]).cross(p
     [(j + 1) % n] - p[j]) > 0) {
    j = (j + 1) % n;
  }
  // Process antipodal pair: p[i] (or
     edge i -> i+1) and p[j]
}
```

2.32 Closest Pair

Description: Finds the closest pair of points. O(nlogn)

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y
     < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P
     (), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(
     v[j++]);
    auto lo = S.lower_bound(p - d), hi =
     S.upper_bound(p + d);
    for (; lo != hi; ++lo) ret = min(ret
     , {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

3 Geometry 3D

3.1 Point 3D

```
using P2 = Point<double>;
struct P3 {
  double x, y, z;
  explicit P3(double x = 0, double y =
     0, double z = 0) : x(x), y(y), z(z)
     {}
  bool operator<(P3 p) const { return
     tie(x, y, z) < tie(p.x, p.y, p.z);
     }
  bool operator==(P3 p) const { return
     tie(x, y, z) == tie(p.x, p.y, p.z);
     }
  P3 operator+(P3 p) const { return P3(x
     + p.x, y + p.y, z + p.z); }
  P3 operator-(P3 p) const { return P3(x
     - p.x, y - p.y, z - p.z); }
  P3 operator*(double d) const { return
     P3(x * d, y * d, z * d); }
  P3 operator/(double d) const { return
     P3(x / d, y / d, z / d); }
  double dot(P3 p) const { return x * p.
     x + y * p.y + z * p.z; }
  P3 cross(P3 p) const { return P3(y * p
     .z - z * p.y, z * p.x - x * p.z, x
     * p.y - y * p.x); }
  double dist2() const { return x * x +
     y * y + z * z; }
  double dist2(P3 p) const { return (*
     this - p).dist2(); }
  double dist() const { return sqrt(
     dist2()); }
  double dist(P3 p) const { return (*
     this - p).dist(); }
  P3 unit() const { return *this / dist
     (); }
  double angle(P3 p) const { return
     atan2(cross(p).dist(), dot(p)); }
  // (this*a).b
  double triple(P3 a, P3 b) const {
     return cross(a).dot(b); }
  // Smallest angle between this and p,
     ignoring direction, in [0, PI/2]
  double small_angle(P3 p) const {
     return acos(min(fabs(dot(p)) / dist
     () / p.dist(), 1.0)); }
  // Same direction, length r
  P3 truncate(double r) const { double k
     = dist(); return !sgn(k) ? *this :
     *this * (r / k); }
  // Rotate vector around unit axis 'u'
     by angle 'a' (Rodrigues' rotation
     formula)
  P3 rotate(P3 u, double a) const {
    u = u.unit();
    return *this * cos(a) + u.cross(*
     this) * sin(a) + u * u.dot(*this) *
     (1 - cos(a));
  }
  // Returns an arbitrary orthogonal
     unit vector to this
  P3 perp() const { return fabs(x) > EPS
     || fabs(y) > EPS ? P3(-y, x, 0).
     unit() : P3(0, -z, y).unit(); }
};
 // orient' is >0 if s lies on lies on
     the same side of plane pqr as (q-p)
     x (r-p), <0 opposite, 0 on it.
 // orient_by_normal: 2D-like
     orientation of p->q->r on the plane
     with normal n.
double orient(P3 p, P3 q, P3 r, P3 s) {
     return (q - p).cross(r - p).dot(s -
     p); }
double orient_by_normal(P3 p, P3 q, P3 r
     , P3 n) { return (q - p).cross(r -
     p).dot(n); }
```

3.2 Plane

```
// 3D Plane: normal n, offset d (n.dot(p
     ) = d)
struct Plane {
  P3 n; double d;
  Plane(P3 n, double d) : n(n), d(d) {}
  Plane(P3 n, P3 p) : n(n), d(n.dot(p))
     {}
  Plane(P3 p, P3 q, P3 r) : Plane((q - p
     ).cross(r - p), p) {}
  double side(P3 p) const { return n.dot
     (p) - d; }
  double dist(P3 p) const { return fabs(
     side(p)) / n.dist(); }
  P3 proj(P3 p) const { return p - n * (
     side(p) / n.dist2()); }
  P3 reflect(P3 p) const { return p - n
     * (2 * side(p) / n.dist2()); }
  // [-PI, PI]
  double angle(Plane p) const { return n
     .angle(p.n); }
  // [0, PI/2]
  double small_angle(Plane p) const {
     return n.small_angle(p.n); }
  bool is_para(Plane p) const { return n
     .cross(p.n).dist() < EPS; }
  bool is_perp(Plane p) const { return
     sgn(n.dot(p.n)) == 0; }
  // Translate the plane by vector t
  Plane translate(P3 t) const { return {
     n, d + n.dot(t)}; }
  // Shift the plane along n by distance
     dis
  Plane shift_up(double dis) const {
     return {n, d + dis * n.dist()}; }
  // Two distinct points lying on the
     plane (n must be non-zero)
  pair<P3, P3> two_pts() const {
    int sx = sgn(n.x), sy = sgn(n.y), sz
     = sgn(n.z);
    assert(sx || sy || sz);
    if (!sx && !sy) return {P3(1, 0, d /
     n.z), P3(0, 1, d / n.z)};
    if (!sy && !sz) return {P3(d / n.x,
     1, 0), P3(d / n.x, 0, 1)};
```

```
    if (!sz && !sx) return {P3(1, d / n.
     y, 0), P3(0, d / n.y, 1)};
    if (!sx) return {P3(1, d / n.y, 0),
     P3(0, 0, d / n.z)};
    if (!sy) return {P3(0, 1, d / n.z),
     P3(d / n.x, 0, 0)};
    if (!sz) return {P3(d / n.x, 0, 1),
     P3(0, d / n.y, 0)};
    if (sgn(d)) return {P3(d / n.x, 0,
     0), P3(0, d / n.y, 0)};
    return {P3(n.y, -n.x, 0), P3(-n.y, n
     .x, 0)};
  }
};
// Intersection line of two planes 3-
    point pairs.
pair<P3, P3> plane_inter(P3 a1, P3 b1,
    P3 c1, P3 a2, P3 b2, P3 c2) {
  auto n1 = (b1 - a1).cross(c1 - a1), n2
     = (b2 - a2).cross(c2 - a2);
  auto d = n1.cross(n2);
  if (d.dist2() == 0) return {P3(), P3()
    };
  auto o = (n2 * n1.dot(a1) - n1 * n2.
    dot(a2)).cross(d) / d.dist2();
  return {o, o + d};
}
```

3.3 Line 3D

```
// 3d line parametric form: `o + k * d`.
struct Line3D {
  P3 d, o;
  Line3D() {}
  Line3D(P3 p, P3 q) : d(q - p), o(p) {}
  Line3D(Plane p1, Plane p2) {
    d = p1.n.cross(p2.n);
    o = (p2.n * p1.d - p1.n * p2.d).
     cross(d) / d.dist2();
  }
  double dist2(P3 p) const { return d.
    cross(p - o).dist2() / d.dist2(); }
  double dist(P3 p) const { return sqrt(
    dist2(p)); }
  P3 proj(P3 p) const { return o + d * (
    d.dot(p - o) / d.dist2()); }
  P3 inter(Plane p) const { return o - d
     * (p.side(o) / d.dot(p.n)); }
  // Orthogonal reflection of point p
    across the line
  P3 reflect(P3 p) const { return proj(p
    ) * 2 - p; }
  // Compare points by their projection
    on the line (use as a sort
    comparator)
  bool cmp_proj(P3 p, P3 q) const {
    return d.dot(p) < d.dot(q); }
  // Small angle between the direction
    vectors of two lines
  double angle(Line3D l) const { return
    d.small_angle(l.d); }
  bool is_para(Line3D l) const { return
    d.cross(l.d).dist() < EPS; }
  bool is_perp(Line3D l) const { return
    sgn(d.dot(l.d)) == 0; }
  // Distance from 3D segment [p, q] to
    point x
  static double seg_dist(P3 p, P3 q, P3
    x) {
    if ((q - p).dot(x - p) < 0) return x
    .dist(p);
    if ((p - q).dot(x - q) < 0) return x
    .dist(q);
    return (q - p).unit().cross(x - p).
    dist();
  }
  // Check if point p lies on segment
    defined by [o, o + d]
  bool on_segment(P3 p) const {
    return dist(p) < EPS && (o - p).dot(
    o + d - p) <= EPS;
  }
};
// Line-line relations: min dist,
    closest point
double line_dist(Line3D l1, Line3D l2) {
  auto n = l1.d.cross(l2.d);
  if (n.dist2() == 0) return l1.dist(l2.
    o);
  return fabs((l2.o - l1.o).dot(n)) / n.
    dist();
}
P3 line_closest(Line3D l1, Line3D l2) {
  auto n2 = l2.d.cross(l1.d.cross(l2.d))
    ;
  return l1.o + l1.d * ((l2.o - l1.o).
    dot(n2) / l1.d.dot(n2));
}
```

3.4 Plane Line

```
// Plane-line angles, relations, and
    perpendiculars through a point.
double angle(Plane p, Line3D l) { return
     PI / 2 - p.n.small_angle(l.d); }
bool is_para(Plane p, Line3D l) { return
     sgn(p.n.dot(l.d)) == 0; }
bool is_perp(Plane p, Line3D l) { return
     p.n.cross(l.d).dist() < EPS; }
Line3D perp_through(Plane p, P3 o) {
    return Line3D(o, o + p.n); }
Plane perp_through(Line3D l, P3 o) {
    return Plane(l.d, o); }
```

3.5 Convex Hull 3d

```
// O(N^2) Incremental 3D Convex Hull
    builder (outputs flattened triangle
    vector)
struct CH3D {
  struct face { int a, b, c; bool ok; };
  int n, num;
  vector<P3> P;
  vector<face> F;
  vector<vector<int>> g;
  void init(vector<P3> p) {
    P = p; n = p.size(); num = 0;
    F.resize(8 * n + 1); g.assign(n + 1,
     vector<int>(n + 1, 0));
  }
  double volume(P3 a, P3 b, P3 c, P3 d)
    { return ((b - a).cross(c - a)).dot
    (d - a); }
  double dblcmp(P3 &p, face &f) { return
     ((P[f.b] - P[f.a]).cross(P[f.c] -
    P[f.a])).dot(p - P[f.a]); }
  void deal(int p, int a, int b) {
    int f = g[a][b]; face add;
    if (F[f].ok) {
      if (dblcmp(P[p], F[f]) > EPS) dfs(
     p, f);
      else {
        add = {b, a, p, true};
        g[p][b] = g[a][p] = g[b][a] =
     num;
        F[num++] = add;
      }
    }
  }
  void dfs(int p, int now) {
    F[now].ok = false;
    deal(p, F[now].b, F[now].a);
    deal(p, F[now].c, F[now].b);
    deal(p, F[now].a, F[now].c);
  }
  void build() {
    if (n < 4) return;
    bool ok = false;
    for (int i = 1; i < n; i++) if (P
     [0].dist(P[i]) > EPS) { swap(P[1],
     P[i]); ok = true; break; }
    if (!ok) return; ok = false;
    for (int i = 2; i < n; i++) if ((P
     [0] - P[1]).cross(P[1] - P[i]).dist
     () > EPS) { swap(P[2], P[i]); ok =
     true; break; }
    if (!ok) return; ok = false;
    for (int i = 3; i < n; i++) if (fabs
     (((P[0] - P[1]).cross(P[1] - P[2]))
     .dot(P[0] - P[i])) > EPS) { swap(P
     [3], P[i]); ok = true; break; }
    if (!ok) return;
    for (int i = 0; i < 4; i++) {
      face add = {(i + 1) % 4, (i + 2) %
     4, (i + 3) % 4, true};
      if (dblcmp(P[i], add) > 0) swap(
     add.b, add.c);
      g[add.a][add.b] = g[add.b][add.c]
     = g[add.c][add.a] = num;
      F[num++] = add;
    }
    for (int i = 4; i < n; i++) {
      for (int j = 0; j < num; j++) if (
     F[j].ok && dblcmp(P[i], F[j]) > EPS
     ) { dfs(i, j); break; }
    }
    int tmp = num; num = 0;
    for (int i = 0; i < tmp; i++) if (F[
     i].ok) F[num++] = F[i];
  }
  // center of mass
  P3 centroid() {
    P3 ans(0, 0, 0), o(0, 0, 0); double
     all = 0;
    for (int i = 0; i < num; i++) {
      double vol = volume(o, P[F[i].a],
     P[F[i].b], P[F[i].c]);
      ans = ans + (o + P[F[i].a] + P[F[i
     ].b] + P[F[i].c]) / 4.0 * vol;
      all += vol;
    }
    return ans / all;
  }
  // Total surface area of the 3D Convex
     Hull
  double area() {
    if (n == 3) return (P[1] - P[0]).
     cross(P[2] - P[0]).dist() * 0.5;
    double res = 0;
    for (int i = 0; i < num; i++) {
      res += (P[F[i].b] - P[F[i].a]).
     cross(P[F[i].c] - P[F[i].a]).dist()
     ;
    }
    return res * 0.5;
  }
  // Total volume enclosed by the 3D
    Convex Hull
  double volume() {
    double res = 0; P3 o(0, 0, 0);
    for (int i = 0; i < num; i++) res +=
      volume(o, P[F[i].a], P[F[i].b], P[
     F[i].c]);
    return fabs(res / 6.0);
  }
  // Are the triangular faces s and t
    coplanar?
  bool same(int s, int t) {
    P3 a = P[F[s].a], b = P[F[s].b], c =
     P[F[s].c];
    return fabs(volume(a, b, c, P[F[t].a
    ])) < EPS && fabs(volume(a, b, c, P
    [F[t].b])) < EPS &&
           fabs(volume(a, b, c, P[F[t].c
    ])) < EPS;
  }
  int num_tri() { return num; } //
    number of surface triangles
  int num_poly() {                //
    number of surface polygons, O(num^2)
    int res = 0;
    for (int i = 0; i < num; i++) {
      bool ok = true;
      for (int j = 0; j < i; j++) if (
     same(i, j)) { ok = false; break; }
      res += ok;
    }
    return res;
  }
  // Distance from point p to the plane
    of face i
  double pt_face_dist(P3 p, int i) {
    return fabs(volume(P[F[i].a], P[F[i
    ].b], P[F[i].c], p)) /
           (P[F[i].b] - P[F[i].a]).cross
    (P[F[i].c] - P[F[i].a]).dist();
  }
  // Check if point p is inside or on
    the 3D Convex Hull
  bool contains(P3 p) {
    for (int i = 0; i < num; i++) {
      if (dblcmp(p, F[i]) > EPS) return
     false;
    }
    return true;
```

```
  }
};
```

3.6 Sphere

```
// Spherical coordinates, distances,
    and sphere-sphere intersections.
P3 sph_pt(double r, double lat, double
    lon) {
  lat *= PI / 180; lon *= PI / 180;
  return P3(r * cos(lat) * cos(lon), r *
     cos(lat) * sin(lon), r * sin(lat))
    ;
}
double great_circle_dist(P3 o, double r,
     P3 a, P3 b) { return r * (a - o).
    angle(b - o); }
struct Sphere {
  P3 c; double r;
  double volume() const { return 4.0 /
    3.0 * PI * r * r * r; }
  double surface_area() const { return 4
     * PI * r * r; }
  // Line-Sphere Intersection Points
  vector<P3> inter(Line3D l) const {
    P3 proj = l.proj(c);
    double d2 = c.dist2(proj);
    if (d2 > r * r + EPS) return {};
    double h = sqrt(max(0.0, r * r - d2)
    );
    P3 dir = l.d.unit();
    if (h < EPS) return {proj};
    return {proj - dir * h, proj + dir *
     h};
  }
  // if this point is inside the sphere
  bool contains(P3 p) const { return sgn
    (p.dist(c) - r) <= 0; }
  // Length of the tangent segment from
    an external point p (-1 if p is
    inside)
  double tangent_len(P3 p) const {
    double d = p.dist(c);
    return sgn(d - r) < 0 ? -1 : sqrt(
    max(0.0, d * d - r * r));
  }
  // Polar angle of the cap of tangency
    points from p (-1 if inside, -2 if
    on the surface)
  double tangent_angle(P3 p) const {
    double d = p.dist(c);
    if (sgn(d - r) < 0) return -1;
    if (sgn(d - r) == 0) return -2;
    return acos(r / d);
  }
};
// sphere-sphere intersection
pair<double, double> sphere_inter(Sphere
     s1, Sphere s2) {
  double d = s1.c.dist(s2.c);
  if (d >= s1.r + s2.r) return {0, 0};
  double R = max(s1.r, s2.r), r = min(s1
    .r, s2.r);
  double x = (R * R - r * r + d * d) /
    (2 * d);
  double h1 = R - x, h2 = (R + r - d) -
    h1;
  double vol = PI * h1 * h1 * (3 * R -
    h1) / 3.0 + PI * h2 * h2 * (3 * r -
    h2) / 3.0;
  double area = 2 * PI * R * h1 + 2 * PI
    * r * h2;
  return {area, vol};
}
// Radius of the circle in which two
    intersecting spheres meet
double sphere_inter_radius(Sphere s1,
    Sphere s2) {
  double d = s1.c.dist(s2.c), R = max(s1
    .r, s2.r), r = min(s1.r, s2.r);
  if (sgn(d - R - r) >= 0) return 0;
  double w = d * d - r * r + R * R;
  return sqrt(max(0.0, 4 * d * d * R * R
    - w * w)) / (2 * d);
}
// Sphere through four non-coplanar
    points
Sphere circum_sphere(P3 a, P3 b, P3 c,
    P3 d) {
  assert(sgn(Plane(a, b, c).side(d)) !=
    0);
  Plane u(a - b, (a + b) / 2), v(b - c,
    (b + c) / 2), w(c - d, (c + d) / 2)
    ;
  Line3D l1(u, v), l2(v, w);
  P3 o = line_closest(l1, l2);
  return {o, o.dist(a)};
}
// Center of the circle through three
    non-collinear coplanar points.
P3 circle_center(P3 a, P3 b, P3 c) {
  P3 v1 = b - a, v2 = c - a;
  double v11 = v1.dot(v1), v22 = v2.dot(
    v2), v12 = v1.dot(v2);
  double base = 0.5 / (v11 * v22 - v12 *
    v12);
  return a + v1 * (base * v22 * (v11 -
    v12)) + v2 * (base * v11 * (v22 -
    v12));
}
// approximate min enclosing sphere
Sphere min_sphere(vector<P3> p) {
  P3 c(0, 0, 0); int n = p.size(), pos =
    0;
  for (auto &i : p) c = c + i;
  c = c / n;
  double ratio = 0.1;
  for (int it = 0; it < 100000; it++) {
    pos = 0;
    for (int i = 1; i < n; i++) if (c.
    dist2(p[i]) > c.dist2(p[pos])) pos
    = i;
    c = c + (p[pos] - c) * ratio;
    ratio *= 0.998;
  }
  return {c, c.dist(p[pos])};
}
```

3.7 TetraHedron Volume

```
// Tetrahedron (A, B, C, D)
double vol_tetrahedron(p3 a, p3 b, p3 c,
     p3 d) {
  return fabs((b - a).cross(c - a).dot(d
    - a)) / 6.0;
}
```